

OpenCV 개념

OpenCV 는 Open Source Computer Vision 의 약자로 영상처리에 사용할 수 있는 오픈 소스 라이브러리이다. OpenCV 에는 영상처리에 필요한 다양한 함수를 제공한다.

자주 사용하는 함수로는 Mat(이미지를 저장하는 변수 형식), Canny : 외곽선 추출, Inrange(이미지 이진화), Resize(이미지 크기 변경), Rectangle, line 등 (이미지에 원하는 모양을 그릴 때 사용함), cvtColor(이미지 표현 형식을 지정함. RGB, Grayscale 등), Erode : (이미지 침식 연산), Dilate(이미지 팽창 연산)

허프변환 검출 함수 : ex) HoughLineP : 이진화 이미지에서 직선을 검출함
등 이 있다. 각 함수들의 사용법은

https://docs.opencv.org/3.4/d7/da8/tutorial_table_of_content_imgproc.html 에서 확인 할 수 있다.

Inrange 함수를 사용하면 특정 범위 내의 색 영역만 추출해서 binary 이미지로 만든다. Binary 이미지는 0 과 1 로만 이루어진 이미지로, 흑백이미지를 말한다. 굳이 색을 추출해서 흑백이미지로 만드는 이유는 컴퓨터가 연산 할 때 RGB 든 HSV 든 3 개의 값을 연산하면 연산 속도가 느려지게 되는데 Binary 이미지를 사용하면 값 0, 1 로만 연산 할 수 있어 연산 시간을 줄일 수 있기 때문이다.

모폴로지란 이진 영상에서 물체의 모양을 깎거나 부풀려서 다듬는 연산이다. 작은 커널을 영상 위에서 한 픽셀씩 옮기며 겹치는 부분을 보고 중심 픽셀 값을 정한다. 필터의 곱셈과 합이 아니라 최솟값/최대값 비교로 동작한다. 모폴로지의 기본적인 연산에는 침식과 팽창이 있고 이 둘을 섞은 열기와 닫기가 있다.

침식

침식은 커널이 덮는 3 * 3 영역이 전부 흰색일 때만 중심 픽셀이 흰색으로 남고, 하나라도 검은색이 있으면 검은색이 된다. 즉, 영역 안의 최솟값을 취하는 것과 같다. 침식 연산을 하면 물체의 바깥쪽이 한 겹 깎이고, 커널보다 작은 점이나 가는 선은 통째로 사라진다.

팽창

팽창은 커널이 덮는 영역에 흰색이 하나라도 있으면 중심이 흰색이 된다. 침식과는 반대로 영역 안의 최대값을 취하는 것과 같다. 팽창 연산을 하면 물체의 바깥쪽이 한 겹 부풀고, 작은 구멍이나 끊어진 틈이 메워진다.

열기

열기는 침식 연산을 한 뒤 팽창 연산을 한 것이다. 열기 연산을 하면 작은 점 노이즈는 제거되고, 큰 물체는 유지된다. 열기가 노이즈만 지우고 물체는 살리는 이유는 침식을 하면 노이즈는 사라지지만 콘도 한겹 줄어들기 때문이다. 콘은 비교적 큰 물체이기 때문에 침식을 적용해도 사라지지 않는다. 이어서 팽창을 하면 이미 사라진 노이즈는 다시 살아나지 않고, 남아 있던 콘만 원래 크기로 돌아오게 되어 열기 연산을 하면 노이즈만 지울 수 있는 것이다. 즉, 깎아서 작은 것들을 지우고, 남은 것을 다시 부풀리는 구조인 것이다.

닫기

닫기는 팽창 연산을 한 뒤에 침식 연산을 진행한다. 닫기 연산을 하면 팽창으로 틈을 메운 뒤, 침식으로 바깥 크기를 원래대로 되돌린다. 이렇게 되면 이미지 내부에 작은 구멍이 있을 때 이를 매우거나 불안전하게 선이 이어져 있을 때 확실하게 이을 수 있다.

이번 과제에서는

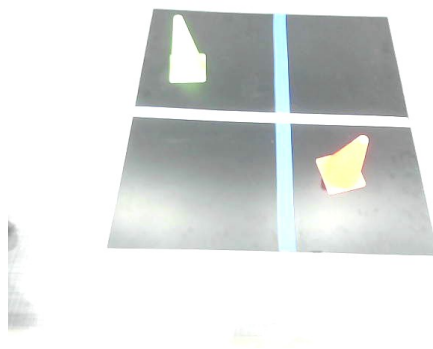
```
cv::morphologyEx(mask, mask, cv::MORPH_OPEN, cv::Mat());
```

열기 연산이 사용되었다. morphology 는 침식/팽창 이외의 조합 연산을 op 인자로 골라 실행하는 함수이다. MORPH_OPEN 을 지정하면 열기 연산을 할 수 있다. (MORPH_ERODE, MORPH_DILATE, MORPH_CLOSE, MORPH_GRADIENT 등도 사용할 수 있다) 입력과 출력에 같은 mask 를 넘겨서 제자리에 덮어 쓴다. 커널 자리에 cv::Mat()을 주면 OpenCV 가 3 * 3 사각형 커널을 자동으로 사용한다. 나머지 인자인 anchor, iterations, BORDER_CONST 는 기본값으로 설정된다. Anchor 는 (-1, -1)로 커널 중심을 기준으로 쓰고, iteration 은 1 회, BORDER_CONST(경계값 처리)는 기본값으로 영상 밖을 정해진 하나의 값으로 채운다는 뜻이다.

리눅스에서는 카메라를 /dev/videoN 파일로 나타낸다. 카메라 한대가 노드 2 개씩 만들고, 그중 하나만 영상 캡처용이다. USB 로 연결한 웹캠을 사용하려면 노드를 다르게 지정해야한다. 카메라를 연결하면 dev/video2 가 새로 생기는 것을 확인하였고, video2 에서 MJPEG 로 입력 받게 설정했다. gui_node 는 /image_raw 를 구독하고, 새 프레임마다 imageCallback 을 호출한다.

ROS 이미지 메시지는 OpenCV 형식이 아니라서 cv_bridge 가 CV::Mat 으로 바꿔준다. 카메라 영상이 RGB 값으로 입력받는데 OpenCV 에서는 BGR 순서를 사용하기 때문에 순서를 바꿔주어야 한다.

처음에 카메라 노드를 켜올 때 밝기 50 에 자동 노출로 시작한다. 이렇게 되면 선이 너무 밝게 인식되어서 파란선도 흰색과 비슷하게 인식되고 주변 반사광도 흰색으로 인식하는 문제가 발생한다.

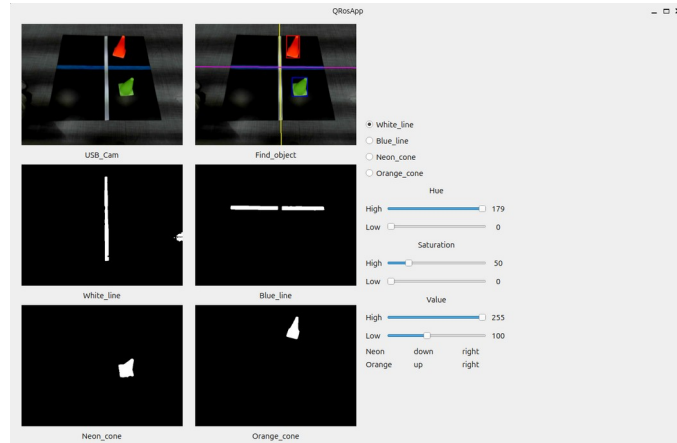


이 문제를 해결하기 위해서는 카메라 설정을 직접해주어야 한다. v4l2-ctl 명령어로 직접 카메라를 설정할 수 있다. brightness 는 밝기 보정, auto_exposure 는 수동, 자동 노출 모드 설정, exposure_time_absolute 는 노출 시간을 설정할 수 있다.

이번 과제에서는 v4l2-ctl -d /dev/video2 -c brightness=0 -c auto_exposure=1 -c exposure_time_absolute=40

이렇게 설정했다. 카메라를 켜올 때 너무 밝은 문제를 노출시간과 기본 밝기의 문제이기 때문에 밝기 보정은 0 으로, 노출시간은 40 으로 설정해 문제를 해결했다.

카메라 노드를 다시켜면 설정이 초기화되므로 노드를 다시 연결했다면 v4l2-ctl 로 다시 설정해주어야 되고, 조명이 바뀌거나 환경이 바뀌면 exposure_time_absolute 로 노출 시간을 조절해주어야한다. 그리고 카메라 노드와 GUI 노드가 같은 값이어야 통신할 수 있다.



코드 설명 :

MainWindowDesign 은 QMainWindow 와 rclcpp::Node 를 동시에 상속한다.

QTimer 가 30ms 마다 rclcpp::spin_some()을 호출해서 쌓인 ROS 콜백을 처리한다.

영상처리는 updateDisplay()가 detector_.detect(lastest_frame_)을 호출하면 아래 순서로 처리된다.

1. 가우시안 블러

```
cv::GaussianBlur(frame, blurred, cv::Size(5, 5), 0);
```

각 픽셀을 주변 5 * 5 픽셀의 가중 평균으로 바꾼다. 중심에 가까울수록 가중치가 커서 테두리가 부드러워지고, 인접 픽셀 간 값이 튀는 노이즈를 제거해서 이후 inRange()에서 저므을 남지 않게 한다. 시그마 값을 0 으로 주면 커널 크기를 자동으로 계산해준다.

2. BGR 값을 HSV 값으로 변환해준다

변환은 cv::cvtColor(blurred, hsv, cv::COLOR_BGR2HSV); 코드를 사용한다.

RGB 값은 같은 색이더라도 밝기가 다르면 RGB 값이 전부 달라져서 색을 감지하고 분류하는데 불리하다. 그래서 이를 HSV 로 바꿔주는 작업이 필요하다. 이렇게 HSV 값으로 데이터를 바꿔주면 V 를 따로 분리해서 “주황색” 을 H 범위 하나로 특정시킬 수 있다. 이렇게 되면 밝기가 변하더라도 색을 감지하기 쉬워진다.

3. inRange 로 마스크 4 장 생성

```
cv::inRange(hsv, Scalar(h_low, s_low, v_low), Scalar(h_high, s_high, v_high), mask);
```

(H,S,V) 세 값이 모두 범위 안이면 255, 하나라도 벗어나면 0 인 1 채널 영상을 만든다. 색상 값들은 다음과 같다.

대상	H	S	V
흰선	0~179	0~50	100~255

대상	H	S	V
파란선	100~120	150~255	50~255
형광콘	35~60	120~255	60~255
주황콘	0~15	150~255	120~255

흰색은 모든 색을 합치면 만들어지기에 채도의 특징을 가지지 않아, H는 전 범위, S는 낮은쪽, V는 높은 쪽으로 설정했다. 색이 있는 대상은 H로 색을 고르고, S/V는 낮게 설정하여 어둡거나 탁한 배경을 걸러낸다.

4. 열기 연산

`cv::morphologyEx(mask, mask, cv::MORPH_OPEN, cv::Mat());`

열기 연산은 침식후에 팽창한다. 침식을 적용하면 흰 픽셀 주면 3 * 3 안에 검은 픽셀이 하나라도 있으면 그 픽셀을 검게 만든다. 이렇게 되면 3 * 3 보다 작은 점이나 가는 선이 사라지면서 나중에 흰색선, 파란색 선을 찾을 때 방해가 되는 노이즈들을 줄일 수 있다. 침식으로 노이즈를 줄인뒤에는 팽창을 적용한다. 팽창은 남은 흰 영역의 가장자리를 3 * 3 만큼 부풀려 큰 물체를 원래 크기로 복원한다.

- `Cv::Mat()`을 커널로 주면 OpenCV가 3 * 3 사각형 커널을 자동으로 사용한다.

5. 덩어리 찾기

`findContours` 로 인접한 1 을 찾아 덩어리로 묶는다.

`cv::findContours(mask, contours, cv::RETR_EXTERNAL, cv::CHAIN_APPROX_SIMPLE);`

서로 이어진 흰 덩어리마다 외곽선을 찾는다. 여기서 `RETR_EXTERNAL` 은 가장 바깥 윤곽만, `CHAIN_APPROX_SIMPLE` 은 직선 구간의 중간 점을 생략한다. `Cv::counterArea` 로 면적을 구해 화면 픽셀 수 대비 일정 비율보다 작으면 노이즈로 처리한다. `findContours` 로 찾은 덩어리중 가장 큰 윤곽을 골라 `cv::boundingRect` 로 감싸는 축 정렬 사각형(x, y, w, h)을 얻는다. 이 사각형이 콘의 위치와 크기를 나타낸다.

6. 흰색 선, 파란색 선 표시

`cv::fitLine(points, line, cv::DIST_L2, 0, 0.01, 0.01);`

점들에 최소제곱법으로 맞는 직선을 구한다. 결과(`vx, vy, x0, y0`)에서 (`vx, vy`)는 선의 방향 단위 벡터, (`x0, y0`)은 선이 지나는 점이다. 사각형이 아니라 직선식이기 때문에 기울어진 선도 표현한다. 흰선은 가장 큰 덩어리 하나의 점만 사용한다. 파란선은 흰선에 가로막혀 두 조각이 되는데, 면적 기준을 설정하고, 면적 기준을 넘는 조각의 점을 모두 합쳐서 하나로 맞춘다. 선의 굵기는 덩어리 면적 + 점들의 바운딩 박스 긴 변으로 근사한다.

7. 흰색선, 파란선 위에 콘을 놓아도 중앙선이 제대로 인식되는 이유

선 위에 콘을 두었을 때 선 위의 선 표시. 시연 영상에서 흰색 선이나 파란색 선 위에 콘을 놓더라도 중앙 선이 뒤틀리거나 선을 벗어나지 않는다. 그 이유는 `fitLine` 이 무한한 직선을 만들기 때문이다. 선이 중간에 끊기더라도 남은 조각으로 구한 직선이 끊긴 구간까지 그대로 이어지고, 위치 판별은 이 직선과 콘 중심 사이의 거리로 구하므로, 콘이 선을 가리고 있어도 거리가 정상적으로 계산된다. 그리고 콘이 선을 가린다고 하더라도 남은 점 만으로 직선을 맞추기 때문에 중간선이 빠지지 않는 것이다.

8. 선으로 x 축, y 축 정하기

|white.vy|와 |blue.vy|를 비교해 더 큰 쪽(y 축에 더 가까운 쪽)을 y 축으로 정한다. y 축은 콘의 좌/우, x 축은 위/아래 기준이 되어 매트릭스 어딘 방향으로 놓아도 콘이 몇사분면에 있는지 일관성있게 분류할 수 있다. 만약 선의 위치가 아닌 색으로 x, y 축을 구별하면 나중에 배드를 돌렸을 때 콘의 위치를 부정확하게 나타낼 수 있기 때문에 이런 방식을 사용했다. ‘

9. 콘의 위치 판정

선 위의 점 (x0, y0)에서 콘 중심(cx, cy)로 가는 벡터를 선의 법선 방향에 투영한 값이 부호 있는 거리 d 이다.

식으로 나타내면 다음과 같다. $d = n_x \cdot (cx - x_0) + n_y \cdot (cy - y_0)$

frame(box)는 원본 Mat 의 사각형 영역을 복사 없이 가르키는 ROI 이고, .clone()으로 독립된 새 Mat 으로 복사한다. 콘이 없으면 빈 Mat 이 반환되어 None 으로 표시된다.

10. 화면 표시 및 GUI 조작

-1) 화면 구성

6 칸: 원본(USB_screen)과 검출 결과(Object_screen L 선과 콘 바운딩 박스), 마스크 4 장

크롬 칸 : 형광콘, 주황콘의 바운딩 박스 내부

위치 라벨 : Neon, Orange 옆에 up/center/down, left/center/right 또는 none 으로 표시한다.

-2) Mat 을 Qt 라벨에 표시

QImage 는 Mat 의 픽셀 메모리를 읽는다. Opencv 가 BGR 이라 Format_BGR888 로 지정해야 색이 뒤집히지 않고, 채널이 1 개인 마스크는 Format_Grayscale8 을 사용한다.

라벨 크기에 맞게 가로새로 비율을 유지해 줄여서 보여준다

라벨은 크기 정책을 Ignored 로 설정한다. 이렇게 설정하지 않으면 그림 크기 때문에 레이아웃이 계속해서 커지는 문제가 발생할 수 있다.

-3) HSV 슬라이더의 동작

1. HSV 범의(한한/상한/ 상한 6 개의 값)는 대상마다 ConeDectector 안에 따로 저장되고 초기값들이 들어 있다.

2.슬라이더 범위는 H 0 ~ 179, S/V 0 ~ 255 이다.

3. 라디오 버튼으로 대상을 바꾸면 onObjectSelected()가 그 대상의 저장된 값을 슬라이더에 채운다. 이때 QsignalBlocker 로 시그널을 잠시 막아, 값을 채우는 동작이 값이 바뀌었다” 는 이벤트로 이어져서 범위를 덮어쓰는 문제를 방지한다.

4. 슬라이더를 움직이면 onSliderChange()가 6 개의 슬라이더의 현재값으로 범위를 만들어 선택된 대상에 저장하고, 옆의 숫자 라벨을 갱신한 뒤에 화면을 다시 그려서 마스크가 즉시 바뀐다.

현재 코드 효과의 한계

3 * 3 커널은 아주 작은 점만을 지운다. 더 큰 노이즈가 발생했을 때는 지우지 못하므로 면적 필터가 필요하다. 열기는 콘 안쪽의 구멍이나 끊김을 메우지는 못한다. 시연에서는 그런 문제가 발생하지는 않았지만, 콘이 반사광 때문에 둘로 나뉘지는 경우에는 닫기를 추가해야 제대로 인식할 수 있다.